

SSO Integration Guide — Frontend Team

How to wire any new Next.js project into the Orchestrator SSO system

Overview

You never build a login page. You never store tokens. The SSO system handles everything:

1. User logs into the **SSO Portal** once (separate app)
 2. SSO Portal shows a list of projects — user clicks your app
 3. Your app opens, checks if the user is authenticated
 4. If not → automatically redirects to Orchestrator → Orchestrator redirects back
 5. Your backend sets an **HttpOnly cookie** (JS can't read it, XSS-proof)
 6. Your frontend sends that cookie automatically on every API call
-

How Authentication Works in Your App



|
/dashboard

└─ redirect back to

|
|
again → 200 

|
└─ useSSO fires

Required Files

1 — Environment variables — `.env.local`

bash

Your project's backend API

`NEXT_PUBLIC_PROJECT_API=http://localhost:8001`

Must end with the root_path of your backend + /sso/authorize

`NEXT_PUBLIC_ORCHESTRATOR_SSO_URL=http://localhost:8000/auth_orchestrator/sso/authorize`

Must match CLIENT_ID registered with the Orchestrator

`NEXT_PUBLIC_CLIENT_ID=proj_myapp_backend`

Must match the /callback route registered with the Orchestrator

`NEXT_PUBLIC_CALLBACK_URL=http://localhost:8001/your_service_name/v1/callback`

2 — SSO Hook — `src/hooks/useSSO.ts`

Create this file. Drop it into **any page that needs authentication**.

typescript

```

"use client";
import { useEffect, useState } from "react";

// — Config from .env —————
const PROJECT_API      = process.env.NEXT_PUBLIC_PROJECT_API!;
const ORCHESTRATOR_SSO  = process.env.NEXT_PUBLIC_ORCHESTRATOR_SSO_URL!;
const CLIENT_ID         = process.env.NEXT_PUBLIC_CLIENT_ID!;
const CALLBACK_URL      = process.env.NEXT_PUBLIC_CALLBACK_URL!;

// The endpoint on YOUR backend that verifies the session cookie
// Change this to match your backend's health/verify endpoint
const SESSION_CHECK_URL = `${PROJECT_API}/your_service_name/v1/dashboard/api/dashboa

// — Types —————
interface AuthState {
  status: "loading" | "authenticated" | "unauthenticated";
  userId?: string;
  error?: string;
}

// — Hook —————
export function useSSO() {
  const [auth, setAuth] = useState<AuthState>({ status: "loading" });

  useEffect(() => {
    const checkAuth = async () => {
      try {
        const response = await fetch(SESSION_CHECK_URL, {
          method: "GET",
          credentials: "include", // ← sends HttpOnly session cookie automatically
        });

        if (response.status === 401) {
          // No session → kick off SSO flow
          const state = Math.random().toString(36).substring(7);

          // Save CSRF state as plain cookie (NOT HttpOnly – backend needs to read it)
          document.cookie = `oauth_state=${state}; path=/; max-age=300`;

          // Full page redirect – browser automatically sends orchestrator_global_session_id
          window.location.href =
            `${ORCHESTRATOR_SSO}?client_id=${CLIENT_ID}&redirect_uri=${CALLBACK_URL}`;
          return;
        }
      } catch {
        // Handle error
      }
    };
    checkAuth();
  });
}

```

```

    }

    if (response.ok) {
      const data = await response.json();

      // Tell AuthGuard that SSO auth succeeded (so other pages don't redirect to login)
      localStorage.setItem("sso_authenticated", "true");

      setAuth({ status: "authenticated", userId: data.user_id });
      return;
    }

    setAuth({ status: "unauthenticated", error: "Unexpected error" });

  } catch {
    setAuth({ status: "unauthenticated", error: "Network error – is the backend up?" });
  }
};

checkAuth();
}, []);

return auth;
}

```

3 — Auth Guard — `src/app/(routes)/guard/AuthGuard.tsx`

The Auth Guard wraps all protected pages. Update it to **skip** pages that handle their own auth via `useSSO`.

typescript

```
"use client";
import { useEffect, useState } from "react";
import { useRouter, usePathname } from "next/navigation";
import { getToken } from "@/utils/storage";

// — Add any page that uses useSSO() here —————
// These pages handle their own auth – the guard must not intercept them
const SSO_HANDLED_ROUTES = ["/dashboard"];

export default function AuthGuard({ children }: { children: React.ReactNode }) {
  const router = useRouter();
  const pathname = usePathname();
  const [loading, setLoading] = useState(true);
  const [token, setTokenState] = useState<string | null>(null);

  // ✅ SSO routes bypass the guard – useSSO handles auth inside the page itself
  if (SSO_HANDLED_ROUTES.includes(pathname)) {
    return <>{children}</>;
  }

  useEffect(() => {
    if (typeof window === "undefined") return;

    const t = getToken();
    const ssoAuth = localStorage.getItem("sso_authenticated"); // set by useSSO on s
    const isAuthenticated = !!t || !!ssoAuth;

    setTokenState(t);

    if (!isAuthenticated) {
      router.replace("/login");
    } else {
      setLoading(false);
    }
  });

  const handleStorageChange = () => {
    const updatedToken = getToken();
    const updatedSso = localStorage.getItem("sso_authenticated");
    if (!updatedToken && !updatedSso) {
      router.replace("/login");
    } else {
      setLoading(false);
    }
  }
}
```

```
};

window.addEventListener("storage", handleStorageChange);
return () => window.removeEventListener("storage", handleStorageChange);
}, [router]);

if (loading) return null;

return token ? <>{children}</> : null;
}
```

4 — Use in your dashboard page — `src/app/(routes)/dashboard/page.tsx`

typescript

```

"use client";
import { useSSO } from "@/hooks/useSSO";
import { Spinner } from "@/components/ui/spinner";

export default function DashboardPage() {
  const auth = useSSO(); // ← add this at the top

  // — These must come BEFORE any other rendering —————
  if (auth.status === "loading") {
    return (
      <div className="flex items-center justify-center h-screen">
        <Spinner />
      </div>
    );
  }

  if (auth.status === "unauthenticated") {
    return (
      <div className="flex items-center justify-center h-screen">
        <p className="text-destructive text-sm">
          {auth.error || "Authentication failed. Please return to the portal and try"}
        </p>
      </div>
    );
  }

  // — auth.status === "authenticated" — render your page normally —————
  return (
    <div>
      {/* your existing dashboard JSX */}
      {/* auth.userId is available if needed */}
    </div>
  );
}

```

5 — API calls — `src/services/apirequest.ts`

Every fetch/axios call to your backend **must include credentials**:

```
typescript
```

```
// ✅ axios
const apiClient = axios.create({
  baseURL: process.env.NEXT_PUBLIC_PROJECT_API,
  withCredentials: true, // ← sends HttpOnly session cookie on every request
});

// ✅ fetch
fetch(`${PROJECT_API}/some-endpoint`, {
  method: "GET",
  credentials: "include", // ← same thing for fetch
});
```

⚠ Without `withCredentials: true` / `credentials: "include"`, the browser strips the session cookie from every request and every call returns 401.

6 — Logout button

```
typescript

function LogoutButton() {
  const handleLogout = () => {
    // Clear the SSO flag
    localStorage.removeItem("sso_authenticated");

    // Redirect to backend logout — it clears the cookie and bounces to Orchestrator
    window.location.href = `${process.env.NEXT_PUBLIC_PROJECT_API}/your_service_name`;
  };

  return (
    <button onClick={handleLogout}>
      Sign Out
    </button>
  );
}
```


Common Mistakes to Avoid

✗ Using `localhost` vs `127.0.0.1` inconsistently

typescript

```
// ✗ Cookie set on localhost:8000 – but you redirect to 127.0.0.1:8000
// Browser treats these as completely different hosts – cookie never sent
window.location.href = "http://127.0.0.1:8000/sso/authorize" // ✗

// ✓ Always use localhost – consistently
window.location.href = "http://localhost:8000/sso/authorize" // ✓
```

✗ Forgetting `credentials: "include"` on the login fetch

typescript

```
// ✗ Cookie returned by server is silently dropped by browser
fetch("/login", { method: "POST", body: ... })

// ✓
fetch("/login", { method: "POST", credentials: "include", body: ... })
```

✗ Using `localStorage` for tokens

typescript

```
// ✗ NEVER do this – tokens in localStorage are readable by any JS (XSS risk)
localStorage.setItem("access_token", token)

// ✓ Tokens live in HttpOnly cookies – set by the backend, invisible to JS
// You never touch the token directly – just send credentials: include
```

✗ Doubling the base path

typescript

```
// If NEXT_PUBLIC_PROJECT_API = "https://domain.com/auth_orchestrator"
// ✗ This results in /auth_orchestrator/auth_orchestrator/list_projects
apiRequest("get", "/auth_orchestrator/list_projects")

// ✓ baseUrl already has the prefix – just use the endpoint
apiRequest("get", "/list_projects")
```

✗ Checking auth before `useSSO` finishes

typescript

```
// ✗ This renders before auth check completes
export default function Page() {
  const auth = useSSO();
  return <Dashboard />;    // renders even when status = "loading"
}

// ✔ Always guard with loading/unauthenticated checks first
export default function Page() {
  const auth = useSSO();
  if (auth.status === "loading") return <Spinner />;
  if (auth.status === "unauthenticated") return <ErrorMessage />;
  return <Dashboard />;    // only renders when status = "authenticated"
}
```

Port Reference (Local Dev)

Port	What
3000	SSO Portal — login + project launcher
8000	Orchestrator — FastAPI backend
4001	Your app — Next.js frontend
8001	Your app — FastAPI backend

Checklist Before Testing

- ☐ `.env.local` has all 4 `NEXT_PUBLIC_*` variables set
- ☐ `NEXT_PUBLIC_CALLBACK_URL` matches exactly what backend registered with Orchestrator
- ☐ `useSSO` is imported and called at the top of your dashboard page
- ☐ Loading and unauthenticated states are handled before rendering the page
- ☐ `withCredentials: true` on axios client (or `credentials: "include"` on every fetch)
- ☐ `SSO_HANDLED_ROUTES` in `AuthGuard` includes `"/dashboard"`

- ☐ `launch_url` in the Orchestrator DB points to your frontend dashboard URL
 - ☐ CORS on your backend allows your frontend's origin
 - ☐ All API calls use the base URL from env — no hardcoded `localhost` in code
-

What You DON'T Need to Build

- ❌ Login page — SSO Portal handles it
- ❌ Token storage — HttpOnly cookies handle it
- ❌ Token refresh logic — backend handles it
- ❌ Auth interceptors that redirect to `/login` on 401 — `useSSO` handles it
- ❌ Decode or inspect JWTs — backend verifies them